

Quicksort deterministico vs randomizzato

Quicksort deterministico.

il **pivot** è **deterministicamente assegnato** (e.g. primo elemento)

→ esiste un **input pessimo** che richiede $O(n^2)$

Quicksort randomizzato.

il **pivot** è **scelto a caso** a ogni passo,

→ **non esiste un caso peggiore:**
il tempo di esecuzione medio (rispetto alla scelta casuale del pivot) è $O(n \log n)$.

Possiamo ora usare **Chernoff** per dimostrare che, **con alta probabilità** $\left(\geq 1 - \frac{1}{n}\right)$,

Randomized Quicksort richiede solo $O(n \lg n)$ confronti (è dato come esercizio sul libro).

Albero della ricorsione

A ogni livello dell'albero viene scelto *casualmente* un nuovo pivot.

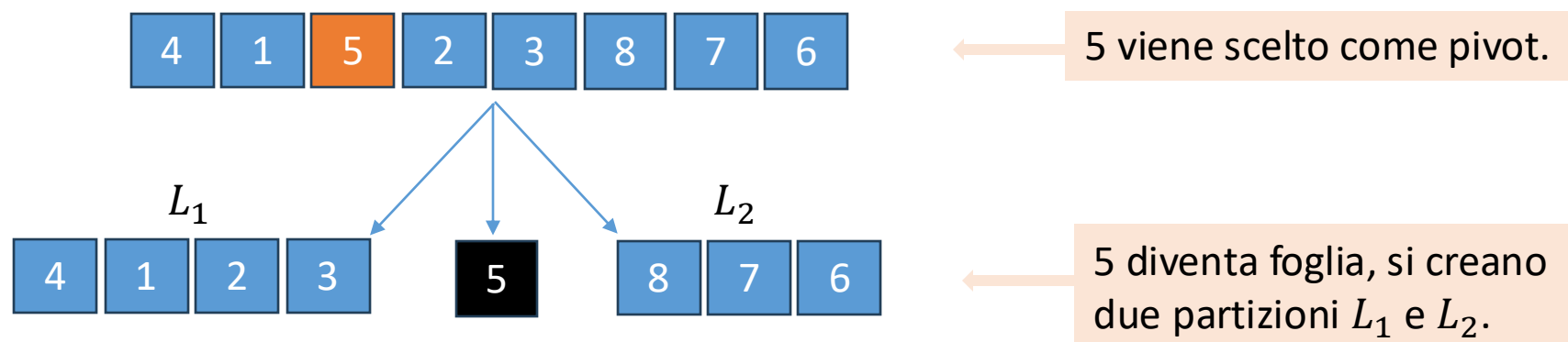
Le sottoliste si riducono progressivamente finché non restano elementi singoli (foglie dell'albero).

È possibile definire **un albero di questo tipo per qualunque esecuzione** di Quicksort randomizzato.

Ogni cammino dalla radice a una foglia rappresenta il percorso seguito da un elemento durante le partizioni successive.

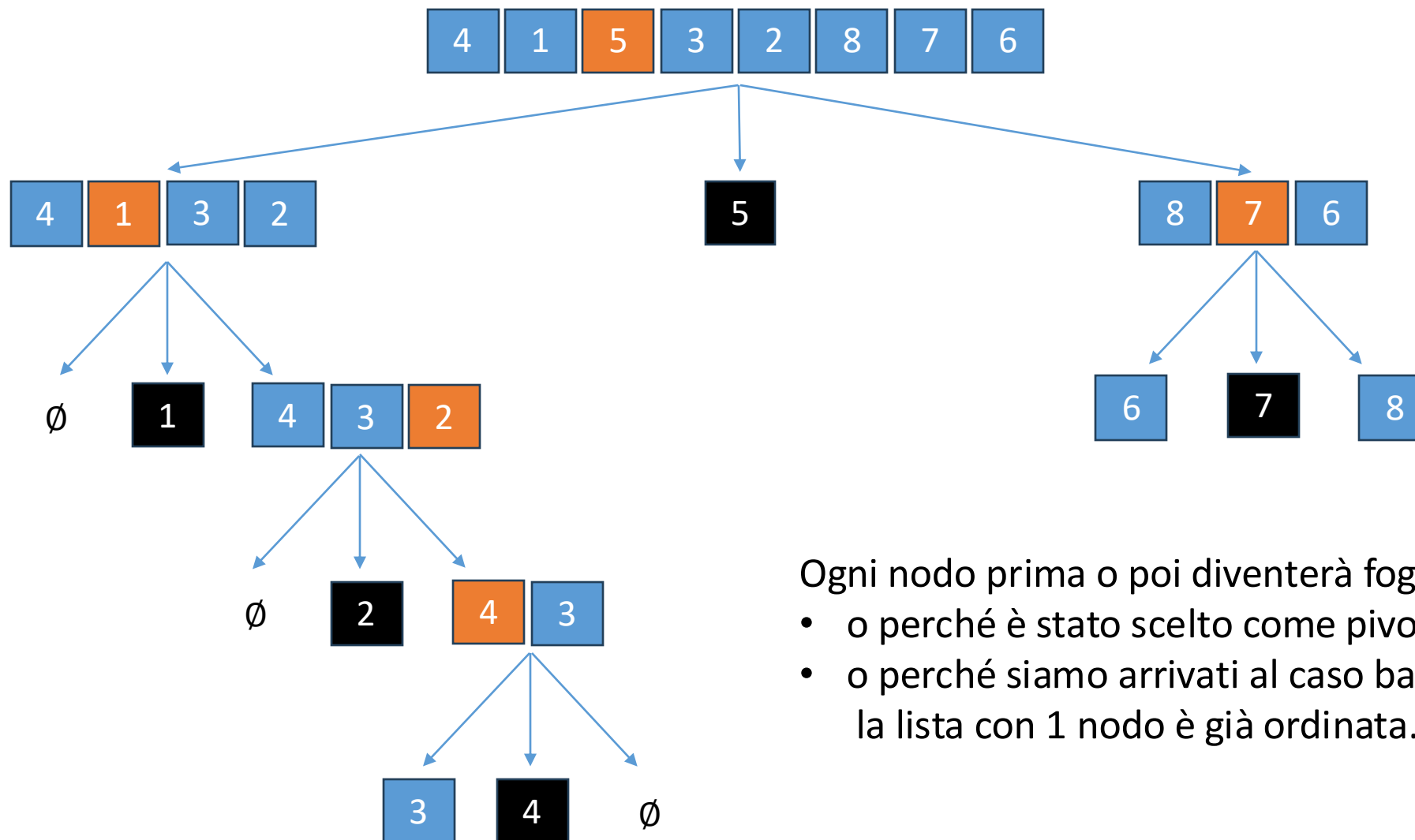
Albero della ricorsione

L'**albero della ricorsione** ha come radice la lista input L . A ogni passo dell'algoritmo, viene scelto il pivot (arancione) e si generano tre figli: sinistro ($<$), centrale (pivot), destro ($>$).



Le sottoliste si riducono progressivamente finché non restano elementi singoli (foglie dell'albero).

Albero della ricorsione



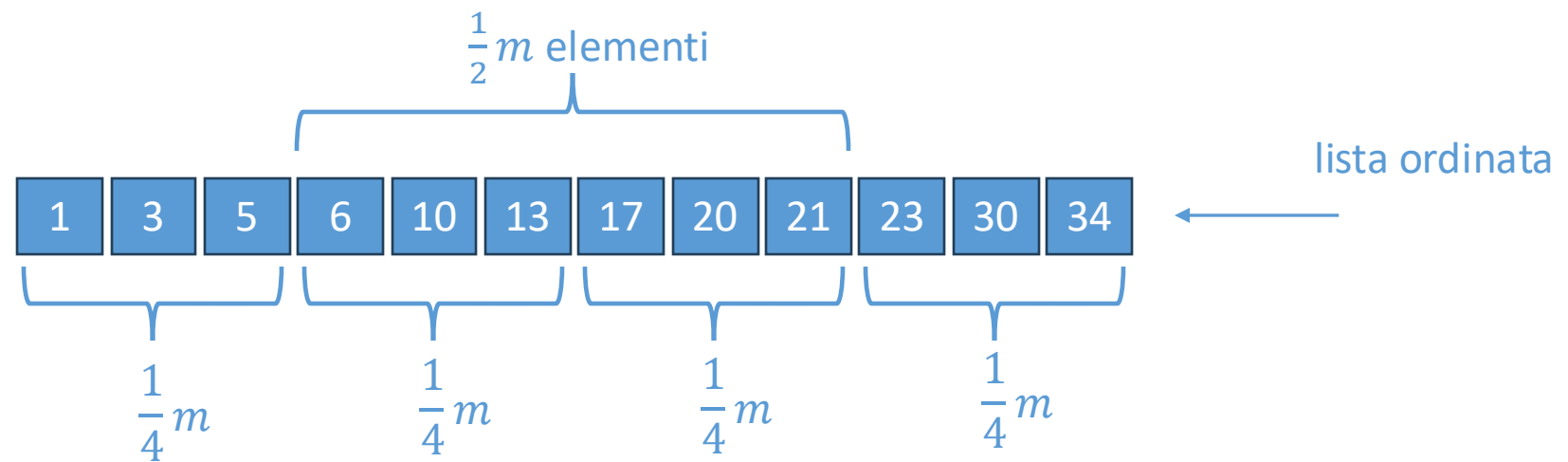
Ogni nodo prima o poi diventerà foglia:

- o perché è stato scelto come pivot;
- o perché siamo arrivati al caso base della ricorsione: la lista con 1 nodo è già ordinata.

Nodi buoni e nodi cattivi

Chiamiamo **buono** un nodo se il pivot separa la lista corrente S (di taglia m) in due sottoliste entrambe di taglia $\leq \frac{3}{4}m$.

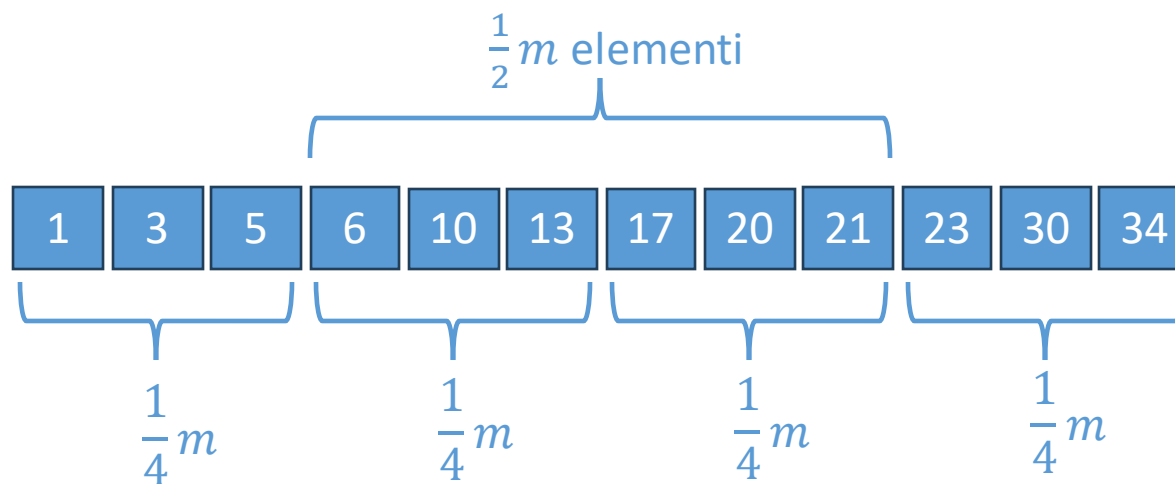
La probabilità che un nodo sia buono corrisponde alla probabilità che il pivot venga scelto nella frazione centrale della lista ordinata, cioè tra l'elemento di rango $\lceil m/4 \rceil$ e quello di rango $\lfloor 3m/4 \rfloor$. Possiamo trascurare $\lceil \cdot \rceil$ e $\lfloor \cdot \rfloor$ perché non influiscono sul limite asintotico.



$$P\{\text{nodo buono}\} \geq \frac{1}{2}$$

Contributo dei nodi buoni

$$P\{\text{nodo buono}\} \geq \frac{1}{2}$$



Ogni nodo *buono* riduce la taglia da m a $\leq \frac{3}{4}m$.

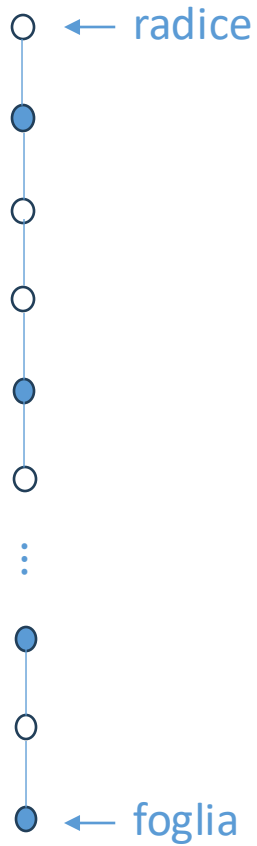
Dopo r nodi buoni la taglia è $\leq (3/4)^r n$.

Per arrivare a taglia 1 (fine ricorsione):

$$(3/4)^r n \leq 1 \Leftrightarrow n \leq (4/3)^r \Leftrightarrow r \geq \log_{4/3} n = \frac{\ln n}{\ln(4/3)}.$$

Contributo dei nodi buoni

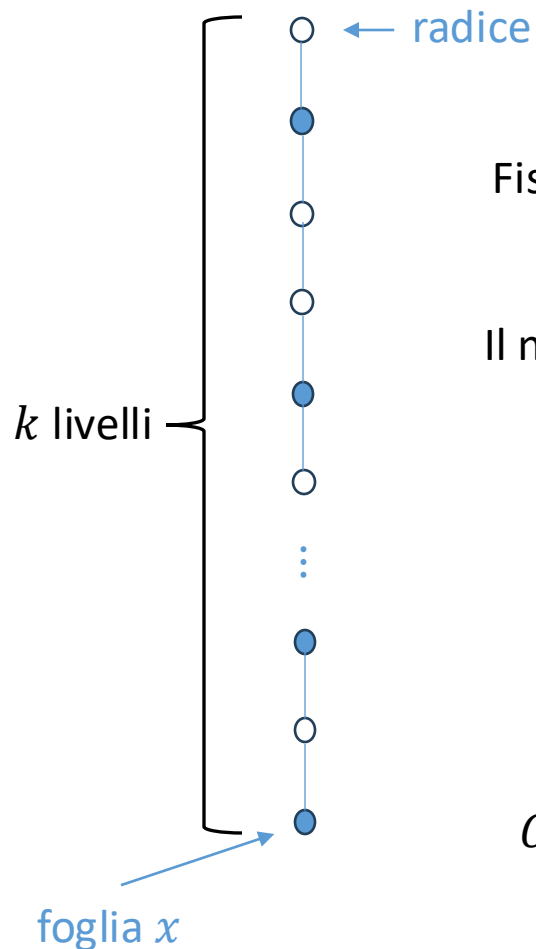
Per arrivare a taglia 1 (fine ricorsione) occorrono $\frac{\ln n}{\ln(4/3)}$ nodi buoni.



Se un cammino ha almeno $\frac{\ln n}{\ln(4/3)}$ nodi buoni,
allora esso contiene sicuramente una foglia.

Contributo dei nodi buoni

Per arrivare a taglia 1 (fine ricorsione) occorrono $\frac{\ln n}{\ln(4/3)}$ nodi buoni.



Fissiamo una foglia x e consideriamo i primi k livelli del suo cammino.

Il numero di nodi buoni entro i primi k livelli è una v.c. $G_k = \sum_{j=1}^k I_j$

dove $I_j = \begin{cases} 1 & \text{se il nodo } j \text{ è buono.} \\ 0 & \text{altrimenti.} \end{cases}$

$G_k \sim \text{Binomiale}(k, p)$ dove $p = \mathbf{P}\{I_j = 1\} \geq \frac{1}{2}$ e $\mu = \mathbf{E}[G_k] = kp \geq k/2$.

Contributo dei nodi buoni

Poniamo $g = \frac{\ln n}{\ln(4/3)}$.

Abbiamo due casi possibili:

- $G_k < g$

entro i primi k livelli il cammino di x **non ha incontrato g nodi buoni**: **non possiamo garantire** che il cammino sia terminato (potrebbe anche essere terminato prima se x è stato scelto come pivot).

- $G_k \geq g$

il cammino **contiene almeno g nodi buoni**, la taglia scende a $(3/4)^g n \leq 1$: quindi x **ha certamente terminato** entro quei k livelli (o prima, se è stato scelto come pivot lungo la strada).

Applichiamo Chernoff

$$\mathbf{P}\{\text{cammino radice-foglia } x \text{ ha lunghezza } \geq k\} \leq \mathbf{P}\{G_k < g\}.$$

$$\mathbf{P}\{G_k < g\} = \mathbf{P}\{G_k - kp < g - kp\} = \mathbf{P}\{G_k - kp < -(kp - g)\} \leq e^{-2(kp-g)^2/k} = \exp\left(-\frac{2}{k}(k/2 - g)^2\right).$$

Scegliamo $k = 6g$. Allora $k/2 - g = 3g - g = 2g$

Chernoff: $\mathbf{P}\{X - kp \leq -\delta\} \leq e^{-2\delta^2/k}$

$$\mathbf{P}\{G_{6g} < g\} \leq \exp\left(-\frac{2(2g)^2}{6g}\right) = \exp\left(-\frac{4}{3}g\right) = n^{-\frac{4}{3 \ln(4/3)}} \leq \frac{1}{n^4}$$

$$\text{Per ogni foglia } x: \mathbf{P}\{\text{cammino radice-foglia } x \text{ ha lunghezza } \geq 6g\} \leq \frac{1}{n^4}$$

Applichiamo l'union bound

Per ogni foglia x : $P\{\text{cammino radice-foglia } x \text{ ha lunghezza} \geq 6g\} \leq \frac{1}{n^4}$

union bound

$$P\{\text{esiste un cammino radice-foglia di lunghezza} \geq 6g\} \leq n \cdot \frac{1}{n^4} = \frac{1}{n^3}$$

$$P\{\text{tutti i cammini radice-foglia sono di lunghezza} < 6g\} > 1 - \frac{1}{n^3}$$

$$P\{\text{numero totale di confronti} < 6n \cdot \frac{\ln n}{\ln(4/3)}\} > 1 - \frac{1}{n^3}$$

Analisi del caso medio

Nell'**analisi nel caso medio**, consideriamo ancora il **Quicksort deterministico**, in cui il pivot è scelto, ad esempio, come **primo elemento** della lista.

Tuttavia, invece di supporre che l'input sia scelto da un avversario, assumiamo che la **lista sia ordinata in modo casuale**.

L'obiettivo è calcolare il **tempo di esecuzione atteso**, dove l'attesa è presa rispetto alla **distribuzione casuale degli input**.

Analisi del caso medio

Poiché l'input è scelto in modo **casuale**, l'avversario **non può controllare** quale sarà il primo elemento di ciascuna sottolista.

Di conseguenza, in ogni fase il **pivot** risulta **effettivamente casuale**, proprio come nel **Quicksort randomizzato**.

Pertanto, il calcolo del **tempo di esecuzione atteso** è identico a quello visto per il Quicksort randomizzato:

$$E[T(n)] = O(n \log n)$$

Analisi del caso medio

Scenario	Scelta del pivot	Tipo di analisi	Tempo di esecuzione
Deterministico – caso pessimo	Primo elemento fisso	Input scelto dall'avversario	$O(n^2)$
Deterministico – caso medio	Primo elemento fisso	Input casuale	$O(n \log n)$
Randomizzato – caso atteso	Pivot scelto a caso	Input arbitrario	$O(n \log n)$

Nel caso medio del Quicksort deterministico, la **casualità è negli input**.

Nel Quicksort randomizzato, la **casualità è nell'algoritmo stesso**.
In entrambi i casi, il **tempo atteso** è $O(n \log n)$.

Attenzione: caso medio $\neq$ valore atteso in generale

È vero che per **Quicksort** i due valori coincidono, ma **non è sempre così** per altri algoritmi.

Differenza concettuale

Analisi

Caso medio (deterministico)

Valore atteso (randomizzato)

Cosa è casuale

L'**input** è casuale

Le **scelte interne** dell'algoritmo (es. pivot) sono casuali

Cosa è fissato

L'**algoritmo** è deterministico

L'**input** è fissato

Attenzione: caso medio $\neq$ valore atteso in generale

Quando coincidono

Coincidono solo se la **distribuzione casuale sugli input** produce lo stesso effetto statistico della **scelta casuale interna** dell'algoritmo.

Esempio: **Quicksort** (pivot casuale o input casuale $\rightarrow$ stesso risultato).

Quando non coincidono

In generale, un algoritmo può avere:

pessimo caso medio se gli input casuali generano strutture sfavorevoli;
ma **buon valore atteso** se la randomizzazione interna “rompe” sistematicamente tali strutture.

k -Select

Definizione del problema

- Input: una lista **non ordinata** di n elementi distinti $a_1, a_2, \dots, a_n$.
- Obiettivo: trovare il **k -esimo elemento più piccolo**.

Per semplicità:

- tutti gli elementi sono distinti

Idea ovvia:

- Ordina la lista (es. con **Randomized Quicksort**)
- Restituisci il **k -esimo elemento** dell'elenco ordinato

Tempo: $O(n \log n)$

k -Select randomizzato

Possiamo ottenere una soluzione **in tempo lineare** $O(n)$?

Idea di base (algoritmo Las Vegas):

Scegliamo un **pivot casuale** (come in *Randomized Quicksort*).

Ma, **a differenza di Quicksort**, qui il pivot ci permetterà di **eliminare parte della lista**, invece di ricorrere su entrambe le metà.

k -Select randomizzato

Dati:

Lista di n elementi, vogliamo trovare il **k -esimo più piccolo**.



Step 1: scegli un **pivot casuale**, $s_i = i$ -esimo elemento più grande

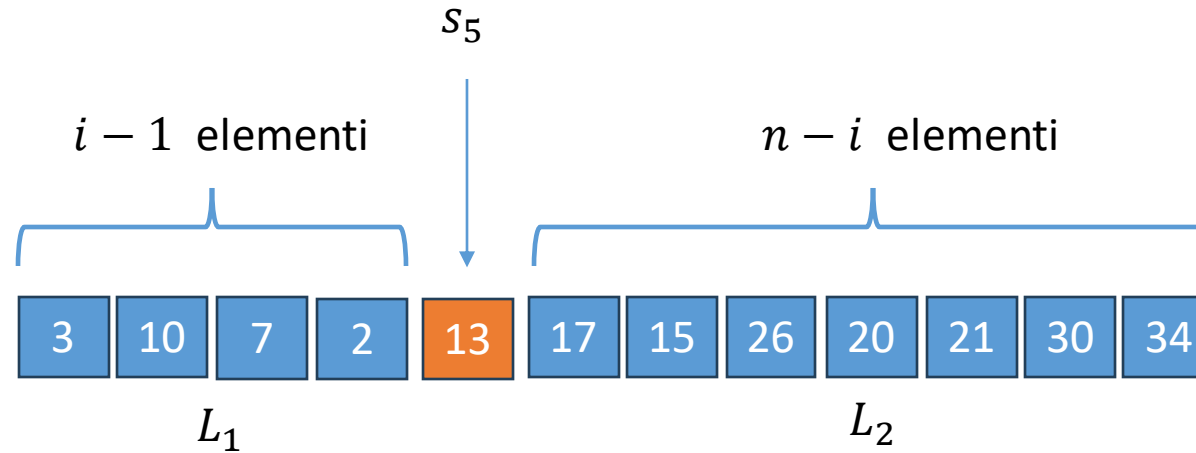
13

Step 2: partiziona la lista in tempo $O(n)$

- L_1 : $i - 1$ elementi **minori** del pivot
- L_2 : $n - i$ elementi **maggiori** del pivot



k -Select randomizzato



Se $k = i$, abbiamo finito.

Domanda: Se $k < i$, a cosa si riduce il nostro problema?

Risposta: A trovare il **k -esimo elemento** in L_1 , che ha dimensione $i - 1$.

Domanda: Se $k > i$, a cosa si riduce il nostro problema?

Risposta: A trovare il **$(k - i)$ -esimo elemento** in L_2 , che ha dimensione $n - i$.

k -Select randomizzato

Teorema 21.2 (Tempo di esecuzione di k -Select randomizzato)

Per ogni lista di n elementi distinti, l'algoritmo **Randomized k -Select** effettua **al più** $c \cdot n$ confronti **in valore atteso**, dove $c = 4$. Questo vale per **qualsiasi valore di k** .

Dim. Sia $C(n)$ il numero di confronti effettuati su una lista di n elementi.

- Effettuiamo $n - 1$ confronti per partizionare la lista attorno al pivot.

Dopodiché:

- Se $k = i$ abbiamo finito.
- Se $k < i$ ricorriamo su L_1 .
- Se $k > i$ ricorriamo su L_2 .

k -Select randomizzato

Teorema 21.2 (Tempo di esecuzione di k -Select randomizzato)

Per ogni lista di n elementi distinti, l'algoritmo **Randomized k -Select** effettua **al più** $c \cdot n$ confronti **in valore atteso**, dove $c = 4$. Questo vale per **qualsiasi valore di k** .

Dim.

$C(n)$ = numero di confronti eseguiti su n elementi.

$$E[C(n)] = \sum_{i=1}^n P\{\text{pivot} = s_i\} E[C(n) \mid \text{pivot} = s_i].$$

Teorema 4.22: valore atteso via condizionamento

$$\leq (n - 1) + \sum_{i=1}^n P\{\text{pivot} = s_i\} E[C(\max(i - 1, n - i))].$$

k -Select randomizzato

Poiché il **pivot** è scelto uniformemente a caso: $P\{\text{pivot} = s_i\} = \frac{1}{n}$

$$\begin{aligned} E[C(n)] &\leq (n-1) + \frac{1}{n} \sum_{i=1}^n E[C(\max(i-1, n-i))] \\ &= (n-1) + \frac{1}{n} \sum_{i=1}^{\lfloor n/2 \rfloor} E[C(n-i)] + \frac{1}{n} \sum_{i=\lfloor n/2 \rfloor+1}^n E[C(i-1)] \\ &= (n-1) + \frac{1}{n} \sum_{i=\lfloor n/2 \rfloor}^{n-1} E[C(i)] + \frac{1}{n} \sum_{i=\lfloor n/2 \rfloor}^{n-1} E[C(i)] \\ &\leq (n-1) + \frac{2}{n} \sum_{i=\lfloor n/2 \rfloor}^{n-1} E[C(i)] \end{aligned}$$

k -Select randomizzato

Ora, per induzione su i dimostriamo che per qualche costante $c \geq 1$:

$$E[C(i)] \leq c \cdot i \quad \text{per ogni } i \leq n.$$

Caso base: per $i = 1$ è ovvio: $E[C(1)] = 0 \leq c \cdot 1$.

Passo induttivo: assumiamo $E[C(i)] \leq c \cdot i$ valida per $i \leq n - 1$.

$$E[C(n)] \leq (n - 1) + \frac{2}{n} \sum_{i=\lceil n/2 \rceil}^{n-1} E[C(i)] \leq (n - 1) + \frac{2}{n} \sum_{i=\lceil n/2 \rceil}^{n-1} c \cdot i = (n - 1) + \frac{2c}{n} \sum_{i=\lceil n/2 \rceil}^{n-1} i.$$

k -Select randomizzato

Somma dei primi m termini di una progressione aritmetica $S_m = a_1 + a_2 + \cdots + a_m$.

$$S_m = \frac{m(a_1 + a_m)}{2}.$$

$$\sum_{i=\lfloor n/2 \rfloor}^{n-1} i = \frac{\lfloor n/2 \rfloor + (n-1)}{2} \cdot ((n-1) - \lfloor n/2 \rfloor + 1) = \frac{\lfloor n/2 \rfloor + (n-1)}{2} \cdot (n - \lfloor n/2 \rfloor).$$

$$\leq \frac{\frac{n}{2} + (n-1)}{2} \cdot \left(n - \frac{n-1}{2} \right) = \frac{\frac{3n}{2} - 1}{2} \cdot \frac{n+1}{2}$$

k -Select randomizzato

$$\begin{aligned} E[C(n)] &\leq (n-1) + \frac{2c}{n} \sum_{i=\lfloor n/2 \rfloor}^{n-1} i = (n-1) + \frac{2c}{n} \cdot \left(\frac{\frac{3n}{2} - 1}{2} \cdot \frac{n+1}{2} \right) \\ &= (n-1) + \frac{c}{n} \cdot \left(\frac{3n-2}{2} \cdot \frac{n+1}{2} \right) \\ &= (n-1) + \frac{c}{4n} \cdot (3n-2) \cdot (n+1) = n-1 + \frac{c}{4n} (3n^2 + n - 2) \\ &= n-1 + \frac{3cn}{4} + \frac{c}{4} - \frac{2c}{4n} \leq n-1 + \frac{3cn}{4} + \frac{c}{4}. \end{aligned}$$

k -Select randomizzato

Ricapitoliamo.

Abbiamo assunto per ipotesi induttiva che $E[C(i)] \leq c \cdot i$ sia valida per $i \leq n - 1$, e abbiamo dimostrato che

$$E[C(n)] \leq n - 1 + \frac{3cn}{4} + \frac{c}{4}.$$

Il nostro obiettivo è far vedere che $E[C(n)] \leq c \cdot n$.

Ci basta trovare il più piccolo c tale che $n - 1 + \frac{3cn}{4} + \frac{c}{4} \leq c \cdot n$.

$$n - 1 \leq \frac{cn}{4} - \frac{c}{4} \quad \text{se e solo se} \quad c \geq 4.$$

k -Select randomizzato

Conclusione.

Per ogni lista di n elementi distinti, l'algoritmo **Randomized k -Select** effettua **al più** $4 \cdot n$ confronti **in valore atteso**. Questo vale per **qualsiasi valore di k** .

Trovare il massimo – Analisi nel caso medio

Dato un array A di lunghezza n contenente interi distinti $a_1, a_2, \dots, a_n$, l'algoritmo **FindMax** determina il valore massimo in A .

Algoritmo — FindMax($a_1, a_2, \dots, a_n$)

1. $\text{currentMax} = -\infty$
2. **for** $i = 1, \dots, n$ **do**
3. **if** ($a_i > \text{currentMax}$) **then** $\text{currentMax} = a_i$
4. **return** currentMax

Assumendo che gli input siano forniti in **ordine casuale uniforme**, quante volte **ci si aspetta** che la variabile currentMax venga aggiornata?

Trovare il massimo – Analisi nel caso medio

Sia X il **numero di volte** in cui currentMax viene aggiornato.

Sia X_i la **variabile indicatrice** dell'evento "l'elemento a_i è il più grande visto finora",
cioè: $a_i > a_j$ per ogni $j < i$.

$\Leftrightarrow X_i$ indica se currentMax viene aggiornato alla posizione i .

Poiché la lista è **in ordine casuale uniforme**, anche l'ordine dei **primi i elementi** è casuale uniforme.

Trovare il massimo – Analisi nel caso medio

Poiché la lista è **in ordine casuale uniforme**, anche l'ordine dei **primi i elementi** è casuale uniforme.

Quindi, la probabilità che **il massimo dei primi i elementi** si trovi in una posizione particolare è

$$P\{X_i = 1\} = \frac{1}{i}.$$

$$E[X] = \sum_{i=1}^n E[X_i] = \sum_{i=1}^n P\{X_i = 1\} = \sum_{i=1}^n \frac{1}{i} = H_n \leq \ln n + 1$$

Verifica randomizzata del prodotto di matrici

Uno degli usi più comuni degli algoritmi randomizzati è la **verifica di correttezza di un programma** (*program checking*).

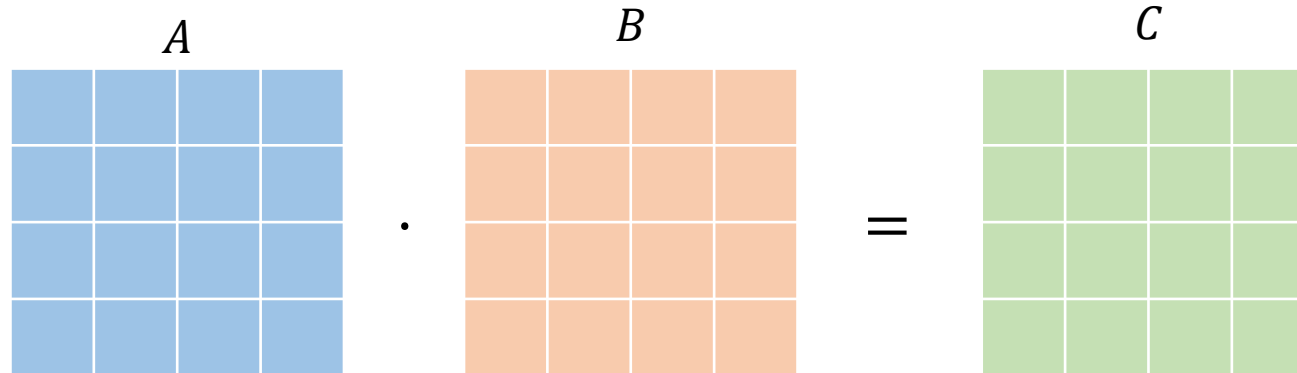
Il caso tipico è quello in cui si dispone di un programma di cui **non ci si fida completamente**.

Si desidera **verificare rapidamente** la correttezza del risultato — in **molto meno tempo** di quanto servirebbe per **ripetere l'intero calcolo da zero**.

Verifica randomizzata del prodotto di matrici

Usando la moltiplicazione standard, per calcolare il prodotto $C = A \cdot B$ di due matrici $n \times n$, A e B occorrono $\Theta(n^3)$ moltiplicazioni.

Ogni elemento di C richiede n moltiplicazioni e ci sono n^2 elementi.



L'algoritmo più efficiente richiede $O(n^{2.371339})$ operazioni (SODA 2025).

Verifica randomizzata del prodotto di matrici

Supponiamo che qualcuno abbia una **versione implementata** dell'algoritmo più efficiente: potremmo **non fidarci del suo codice (perché l'algoritmo è di difficile implementazione)**.

Vorremmo allora **verificare l'esattezza di ogni risultato** che l'implementazione produce. Cioè, ogni volta che si forniscono due matrici A e B e l'algoritmo restituisce C , vogliamo **controllare se davvero $A \cdot B = C$** .

Naturalmente, potremmo calcolare $A \cdot B$ direttamente per verificare, ma ciò richiederebbe ancora tempo $\Theta(n^3)$, **vanificando il vantaggio** dell'algoritmo più veloce.

Algoritmo di Freivalds

Mostreremo ora un **algoritmo Monte Carlo dovuto a Freivalds (1977)** che permette di verificare se $A \cdot B = C$ usando soltanto $\Theta(n^2)$ moltiplicazioni.

L'idea si basa sulla semplice osservazione seguente.

Per **ogni** vettore $\mathbf{r} = (r_1, r_2, \dots, r_n)$, deve valere

$$A \cdot B \cdot \mathbf{r} = C \cdot \mathbf{r}.$$



Proprietà associativa
del prodotto matriciale



$$A \cdot (B \cdot \mathbf{r}) = C \cdot \mathbf{r}.$$